Quasi-Monte Carlo: Foundations

Quant Finance Portfolio — Phase 2, Block 3.0

May 6, 2026

Abstract

We introduce the framework of quasi-Monte Carlo (QMC) integration: deterministic sequences of points designed to fill the unit cube more uniformly than random samples, achieving asymptotic error $O((\log n)^d/n)$ in dimension d versus the standard $O(1/\sqrt{n})$ of Monte Carlo. The price is the loss of statistical machinery (no central limit theorem, no confidence intervals); we recover this via randomized QMC (RQMC) with digital shifting. We develop the discrepancy framework, prove the Koksma-Hlawka inequality, and construct two foundational sequences: Halton (an extension of van der Corput's one-dimensional sequence) and Sobol (based on primitive polynomials over $\mathbb{F}_2$). The relation to inverse sampling — which we adopted in Block 0 of this phase specifically for QMC compatibility — is highlighted throughout.

Contents

1 From Monte Carlo to quasi-Monte Carlo **1**
1.1 The asymptotic comparison 2

2 The van der Corput sequence **3**

3 The Halton sequence **3**

4 The Sobol sequence **3**
4.1 Direction numbers and the recursion 4
4.2 The Joe-Kuo tables 4
4.3 The Gray-code optimisation 4

5 Quasi-Monte Carlo with inverse sampling **4**

6 The need for randomization: from QMC to RQMC **5**
6.1 Digital shifting 5
6.2 The RQMC estimator 5
6.3 The trade-off in choosing n vs R 6

7 Summary and look ahead to Block 3.1 **6**

8 Bibliographic notes **6**

1 From Monte Carlo to quasi-Monte Carlo

The Monte Carlo identity is

$$\int_{[0,1]^d} f(u)\,\mathrm{d}u \;\approx\; \frac{1}{n}\sum_{i=1}^{n} f(u_i), \qquad u_i \sim \mathrm{Unif}\big([0,1]^d\big) \text{ i.i.d.}, \tag{1}$$

with statistical error of order $\sigma(f)/\sqrt{n}$ where $\sigma^2(f) = \mathrm{Var}(f(u))$. The variance reduction techniques of Block 2 attack the constant $\sigma(f)$; they leave the rate $1/\sqrt{n}$ untouched.

QMC takes a different route. Instead of choosing $\{u_i\}$ randomly, we choose them deterministically to be "as uniform as possible" in $[0,1]^d$. The hope is that the average $\frac{1}{n}\sum f(u_i)$ converges to the integral faster than $1/\sqrt{n}$.

The challenge is to make "as uniform as possible" precise. The notion is captured by the *discrepancy* of a point set, which measures the worst-case deviation of the empirical distribution from the uniform distribution on rectangles.

Definition 1.1 (star discrepancy)**.** For a finite point set $P = \{u_1, \ldots, u_n\} \subset [0,1)^d$ and a rectangle $B = \prod_{j=1}^d [0, b_j) \subset [0,1)^d$, let

$$\mathrm{D}^*(P; B) \;=\; \left| \frac{\#\{i : u_i \in B\}}{n} - \mathrm{vol}(B) \right|. \tag{2}$$

The *star discrepancy* of P is

$$\mathrm{D}^*(P) \;=\; \sup_{B \subset [0,1)^d} \mathrm{D}^*(P; B). \tag{3}$$

A sequence with small discrepancy distributes points evenly across all anchored rectangles. For random points, $\mathbb{E}[\mathrm{D}^*(P)] = \Theta(\sqrt{(\log\log n)/n})$ by the law of the iterated logarithm; for QMC sequences, deterministic constructions can achieve $\mathrm{D}^*(P) = O((\log n)^d/n)$.

Theorem 1.2 (Koksma-Hlawka inequality)**.** *For any function $f : [0,1]^d \to \mathbb{R}$ of bounded variation in the sense of Hardy-Krause, with variation $V(f)$, and for any finite point set $P \subset [0,1)^d$,*

$$\left| \frac{1}{n} \sum_{i=1}^{n} f(u_i) \;-\; \int_{[0,1]^d} f(u)\, \mathrm{d}u \right| \;\le\; V(f) \cdot \mathrm{D}^*(P). \tag{4}$$

The Koksma-Hlawka bound is *deterministic*: no probability is involved. Its strength is that, for nice f (smooth, bounded mixed partial derivatives), $V(f) < \infty$, and the integration error decays as fast as the discrepancy. Its weakness, on which we return below, is that $V(f)$ is typically very hard to bound (and often infinite for the indicators that appear in option payoffs), so the inequality gives no usable a priori error estimate.

1.1 The asymptotic comparison

For Halton or Sobol sequences in dimension d, the discrepancy satisfies $\mathrm{D}^*(P_n) = O((\log n)^d/n)$. Compared to MC:

Method	Error rate	Behaviour as $n \to \infty$
MC	$O(n^{-1/2})$	always wins eventually
QMC, d moderate	$O((\log n)^d/n)$	wins MC for moderate n
QMC, d large	$O((\log n)^d/n)$	loses to MC for practical n

The crossover point depends on d. For $d \le 5$, QMC dominates MC at any $n \ge 10^3$. For $d = 30$, the $(\log n)^d$ factor is catastrophic — $(\log 10^6)^{30} \approx 10^{34}$ — and QMC requires extreme n to outperform MC. In practice, QMC is most useful for low-to-moderate dimensional problems.

A modern viewpoint, developed by Sloan, Woźniakowski, Owen and others, refines this picture: many financial integrands, despite high nominal dimension, have "effective dimension" much smaller than d because the function depends primarily on a low-dimensional subspace. QMC then performs well even at large nominal d. This is beyond the scope of the present writeup but motivates the preparatory experiments of Block 3.1.

2 The van der Corput sequence

The simplest QMC construction is one-dimensional. Given an integer base $b \geq 2$, write each non-negative integer i in base b as

$$i \;=\; \sum_{k\geq 0} a_k(i)\, b^k, \qquad a_k(i) \in \{0, 1, \ldots, b-1\}. \tag{5}$$

The *radical inverse function* $\phi_b : \mathbb{N} \to [0, 1)$ reverses the digits and places a decimal point:

$$\phi_b(i) \;=\; \sum_{k\geq 0} a_k(i)\, b^{-(k+1)}. \tag{6}$$

The *van der Corput sequence in base* b is $\{\phi_b(i)\}_{i\geq 1}$.

Example 2.1 (base 2). The base-2 van der Corput sequence starts

$$\tfrac{1}{2},\ \tfrac{1}{4},\ \tfrac{3}{4},\ \tfrac{1}{8},\ \tfrac{5}{8},\ \tfrac{3}{8},\ \tfrac{7}{8},\ \tfrac{1}{16},\ \tfrac{9}{16}, \ldots$$

After 2^k points, the sequence has used each "odd numerator over 2^{k+1}" exactly once, so the points are perfectly equidistributed in $[0, 1)$ at scale $2^{-(k+1)}$.

The van der Corput sequence in base b has discrepancy $\mathrm{D}^* = O(\log n/n)$, optimal in $d = 1$.

3 The Halton sequence

The natural multi-dimensional extension is to use a different prime base in each coordinate. Choose primes $p_1 < p_2 < \cdots < p_d$ (e.g., $p_1 = 2, p_2 = 3, p_3 = 5, \ldots$), and define the Halton sequence by

$$\mathbf{u}_i \;=\; \big(\phi_{p_1}(i), \phi_{p_2}(i), \ldots, \phi_{p_d}(i)\big) \in [0, 1)^d. \tag{7}$$

The construction is constructive, infinite, and trivial to implement: for each new index i, evaluate d radical inverses.

The Halton sequence has discrepancy $\mathrm{D}^* = O((\log n)^d/n)$. The constant in the big-O grows with d: more precisely, the asymptotic constant is $\prod_{j=1}^{d} \frac{p_j - 1}{2 \log p_j}$, which for p_d around 20 (i.e., $d \approx 8$) becomes uncomfortably large.

The high-dimensional pathology. Halton's reliance on primes that grow with dimension introduces a well-known artefact: for $d \geq 7$, projections of the Halton sequence onto pairs of high-index dimensions exhibit visible linear correlations. The prime $p_8 = 19$ is large enough that the radical inverse "advances" very slowly within a window of consecutive indices, producing alignment with $p_9 = 23$. This is why Sobol, despite being more complex to construct, is preferred in practice.

Implementation. The radical inverse is computed iteratively: while $i > 0$, extract the lowest digit of i in base b, multiply by $b^{-(k+1)}$, accumulate. The whole Halton sequence in dimension d requires $O(d \log_2 n)$ digit operations per point.

4 The Sobol sequence

Sobol sequences are the standard choice for QMC in finance. Whereas Halton uses a different prime base per dimension, Sobol uses base 2 in every dimension and varies a different ingredient: the *direction numbers*, which encode primitive polynomials over $\mathbb{F}_2$.

4.1 Direction numbers and the recursion

For each dimension j, fix a sequence of *direction numbers* $v_1^{(j)}, v_2^{(j)}, \ldots$ where each $v_k^{(j)} \in (0,1)$. The Sobol value for index i in dimension j is

$$u_i^{(j)} \;=\; b_1(i)\, v_1^{(j)} \oplus b_2(i)\, v_2^{(j)} \oplus \cdots \oplus b_K(i)\, v_K^{(j)}, \tag{8}$$

where $b_k(i)$ are the binary digits of the index i ($i = \sum_k b_k(i) 2^{k-1}$), K is the number of bits needed to represent i, and $\oplus$ is bitwise XOR (interpreted as addition modulo 2 on the binary expansion of $v_k^{(j)}$).

For the construction to produce a low-discrepancy sequence, the direction numbers must satisfy specific algebraic properties. They are derived from a primitive polynomial over $\mathbb{F}_2$ for each dimension; the construction is due to Sobol (1967) and refined by Joe and Kuo (2008) into a tabulated form.

4.2 The Joe-Kuo tables

Joe and Kuo published direction-number tables for dimensions up to 21 201 in their 2008 paper. The tables are public and freely distributed at the URL web.maths.unsw.edu.au/∼fkuo/sobol/. They are the standard reference used by virtually all production implementations of Sobol in finance (including `scipy.stats.qmc.Sobol`, which loads them at import time).

The structure of the table is, for each dimension d:

- A primitive polynomial degree s and an integer encoding a representing the polynomial coefficients.
- Initial direction numbers $m_1, m_2, \ldots, m_s$ (one per degree-of-freedom of the polynomial).

The remaining direction numbers are generated by the recursion

$$m_k \;=\; 2a_1 m_{k-1} \oplus 2^2 a_2 m_{k-2} \oplus \cdots \oplus 2^{s-1} a_{s-1} m_{k-s+1} \oplus (2^s m_{k-s} \oplus m_{k-s}), \quad k > s, \tag{9}$$

where a_j are the polynomial coefficients (with $a_0 = a_s = 1$ by convention). The direction numbers $v_k = m_k / 2^k$ are then inserted in equation (8).

The construction may look intimidating; in practice, it is ~ 50 lines of code once the Joe-Kuo file is loaded.

4.3 The Gray-code optimisation

A naive implementation evaluates the XOR over K direction numbers for each new index i. Sobol observed that consecutive indices in *Gray-code order* differ in exactly one bit, so generating u_{i+1} from u_i requires only one XOR. This optimisation is essential for performance; we adopt it in the C++ implementation.

5 Quasi-Monte Carlo with inverse sampling

Suppose we want to estimate $\mathbb{E}[g(Z)]$ where $Z \sim \mathcal{N}(0,1)^d$. The MC route is:

1. Draw $u_i \sim \mathrm{Unif}([0,1)^d)$ i.i.d.
2. Apply $z_i = \Phi^{-1}(u_i)$ componentwise.
3. Average $g(z_i)$.

The QMC route replaces step 1 by a low-discrepancy sequence and keeps steps 2-3 unchanged. *Crucially, this only works if the mapping Φ^{-1} is monotone and one-to-one.* Other normal samplers — Box-Muller (which mixes pairs of uniforms in a non-monotone way), Marsaglia polar (which involves rejection), Ziggurat (rejection-based) — destroy the low-discrepancy structure of the input. Box-Muller, in particular, can introduce spurious correlations between dimensions when paired with a QMC source.

This is why we adopted inversion sampling in Block 0 of this phase, even though Ziggurat is faster for plain MC. The pipeline

$$\text{QMC source} \longrightarrow \Phi^{-1} \longrightarrow \text{normal samples} \tag{10}$$

preserves the discrepancy structure across the transformation. The choice of normal sampler made in Block 0 was deliberately taken with this future need in mind.

6 The need for randomization: from QMC to RQMC

Consider the situation after running a QMC integrator. We have computed

$$\hat{I}_n^{\mathrm{QMC}} = \frac{1}{n}\sum_{i=1}^{n} f(u_i), \tag{11}$$

a single deterministic number. The Koksma-Hlawka bound gives a deterministic error estimate $|I - \hat{I}_n^{\mathrm{QMC}}| \le V(f)\,\mathrm{D}^*$, but $V(f)$ is essentially never available: the Hardy-Krause variation of an option payoff is hard to bound, often infinite for indicator functions, and not available in closed form even for the European call.

We have a number with no error bar. This is unacceptable in production: pricing requires a quantifiable uncertainty.

6.1 Digital shifting

The fix is to randomise the QMC sequence in a way that preserves its low-discrepancy property but makes each marginal point uniform. The simplest randomisation is the *digital shift.*

Definition 6.1 (digital shift)**.** Given a base-2 QMC sequence $\{u_i\}$ in $[0,1)^d$ and a uniform random shift $\xi \in [0,1)^d$, the digitally shifted sequence is

$$\tilde{u}_i^{(j)} = u_i^{(j)} \oplus \xi^{(j)} \quad \text{(bitwise XOR)}. \tag{12}$$

For each fixed i and j, $\tilde{u}_i^{(j)} \sim \mathrm{Unif}([0,1))$ marginally; the dependence structure between indices is preserved.

The digital shift is reversible (XOR with ξ twice gives back the original), preserves the local properties of Sobol points (by linearity of XOR over $\mathbb{F}_2$), and makes each individual $\tilde{u}_i$ marginally uniform.

6.2 The RQMC estimator

Definition 6.2 (RQMC estimator with R replications)**.** Choose $R \ge 2$ replications. For each replication $r = 1, \ldots, R$, draw an independent random shift $\xi^{(r)}$ and compute

$$\hat{I}_n^{(r)} = \frac{1}{n}\sum_{i=1}^{n} f(u_i \oplus \xi^{(r)}). \tag{13}$$

The RQMC estimator is the average across replications,

$$\hat{I}_{n,R}^{\mathrm{RQMC}} = \frac{1}{R}\sum_{r=1}^{R} \hat{I}_n^{(r)}. \tag{14}$$

The R replication estimates $\{\hat{I}_n^{(r)}\}_{r=1}^R$ are i.i.d., so we have a CLT for them. The RQMC estimator's variance is σ^2_{rep}/R where $\sigma^2_{\text{rep}} = \text{Var}(\hat{I}_n^{(r)})$, and the half-width is

$$h_{n,R}^{\text{RQMC}} \;=\; z_{\beta/2} \cdot \frac{\hat{\sigma}_{\text{rep}}}{\sqrt{R}}, \tag{15}$$

identical in form to the MC half-width but with the variance estimated from R replication-estimates instead of from n payoffs.

The crucial empirical fact: σ^2_{rep} scales with the QMC error, so for nice integrands,

$$\sigma^2_{\text{rep}} \;=\; O\big((\log n)^{2d}/n^2\big) \quad \text{instead of} \quad O(\sigma^2(f)/n). \tag{16}$$

At fixed total budget nR, RQMC produces estimator variance asymptotically smaller than MC by orders of magnitude (for nice f and moderate d).

6.3 The trade-off in choosing n vs R

For a fixed total budget $N = nR$ payoff evaluations:

- Larger n exploits the QMC convergence rate.
- Larger R gives more replications and a tighter half-width estimate (improved by $1/\sqrt{R}$ in the CLT sense, but also useful for stability of $\hat{\sigma}_{\text{rep}}$).

The standard practical rule is $R \in [10, 30]$ replications and the remaining budget on n. We adopt $R = 20$ in the validation script of Block 3.1.

7 Summary and look ahead to Block 3.1

We have established the framework:

- QMC sequences (Halton, Sobol) achieve discrepancy $O((\log n)^d/n)$, faster than the MC rate $O(\sqrt{(\log\log n)/n})$.
- The Koksma-Hlawka bound is deterministic but practically unusable for option payoffs because $V(f)$ is hard to bound.
- Inverse sampling (chosen in Block 0) is the only normal sampler that preserves QMC structure.
- RQMC with digital shifting recovers a usable error bar by averaging R independent shifts.

Block 3.1 specialises this framework to the European call. We will implement Halton and Sobol generators, build a multi-step Euler MC pricer with $d = 20$ (one normal per discretisation step), compare deterministic-QMC and RQMC against the IID Block 1.2.1 baseline, and measure empirical convergence rates on a log-log slope plot.

8 Bibliographic notes

The standard reference for QMC integration is Niederreiter's monograph [4]. The original Sobol sequence is from [6]. The Joe-Kuo tables and primitive polynomials construction are in [2]; the table file `new-joe-kuo-6.21201` is hosted at [3]. Owen's work on RQMC, including the more sophisticated nested scrambling, is reviewed in [5]. The financial-applications viewpoint, including effective dimension, is developed by Glasserman [1, Chapter 5].

References

[1] P. Glasserman, *Monte Carlo Methods in Financial Engineering*, Springer, 2004.

[2] S. Joe and F. Y. Kuo, *Constructing Sobol' sequences with better two-dimensional projections*, SIAM Journal on Scientific Computing, 30(5):2635–2654, 2008.

[3] S. Joe and F. Y. Kuo, *Sobol' sequence generator*, `https://web.maths.unsw.edu.au/~fkuo/sobol/`.

[4] H. Niederreiter, *Random Number Generation and Quasi-Monte Carlo Methods*, SIAM, 1992.

[5] A. B. Owen, *Quasi-Monte Carlo sampling*, in *Monte Carlo Ray Tracing*, ACM SIGGRAPH course notes, 2003.

[6] I. M. Sobol', *On the distribution of points in a cube and the approximate evaluation of integrals*, USSR Computational Mathematics and Mathematical Physics, 7(4):86–112, 1967.